

INGEGNERIA DEL SOFTWARE · INGEGNERIA
INFORMATICA · UNISA

Modelli di Processo

Appunti di ripasso e materiale d'esame — Lezione 2

Processo software

Ruoli & attività

Waterfall

Modello incrementale

IA generativa & SE

Fonte del corso

Slide "Lezione 2 — Modelli di Processo", P. Ritrovato — N. Capuano, DIEM, Università di Salerno.

Testo di riferimento: I. Sommerville, *Ingegneria del Software*, 10^a ed., Pearson, Capitolo 2 (tranne 2.3 e 2.4).

Rielaborato e ampliato per lo studio e la preparazione dell'esame.

Indice

1. Introduzione ai modelli di processo

Cos'è un processo software · Ruoli · Attività (task) · Linee guida · A cosa serve un processo esplicito · Cos'è un modello di processo

2. Modelli di processo tradizionali

Code-and-fix model · Le attività principali (requisiti, design, implementazione, testing, rilascio/manutenzione) · Costo delle attività · Modello a cascata (Waterfall) · Varianti con feedback · Modelli iterativi · Modello incrementale

3. IA Generativa e Processo di Sviluppo Software

No Silver Bullet · Complessità accidentale vs essenziale · IA-Assisted Software Engineering · Debito tecnico e cognitivo · Il ruolo dell'ingegnere

4. Riepilogo per l'esame

Mappa concettuale in breve · Domande di autoverifica con risposta

1 · Introduzione ai modelli di processo

1.1 Cos'è un processo software

DEFINIZIONE Un **Processo di Produzione del Software** (o **Processo Software**) fornisce un insieme di regole che consentono di svolgere, coordinare e controllare in maniera **sistematica** le attività necessarie per raggiungere gli obiettivi di un progetto software.

Un processo software, per essere completo, deve definire cinque elementi:

- Un insieme di **attività** che devono essere svolte
- Gli **input** e gli **output** di ciascuna attività
- Le condizioni per **avviare** l'attività (*entry conditions*)
- Le condizioni per considerare l'attività **completata** (*exit conditions*)
- Un insieme di **ruoli** che devono essere ricoperti dalle persone che partecipano
- Delle **linee guida** sullo svolgimento del progetto

PERCHÉ SERVONO ENTRY/EXIT CONDITION Senza condizioni di ingresso e di uscita ben definite, diventa impossibile stabilire con certezza "a che punto siamo" nel progetto: un'attività potrebbe iniziare troppo presto (con input incompleti) o essere dichiarata finita quando in realtà mancano ancora dei controlli. Le entry/exit condition sono quindi lo strumento con cui il processo rende oggettivo l'avanzamento del lavoro, invece di lasciarlo a valutazioni soggettive come "ci siamo quasi".

1.2 Ruoli

Un progetto software coinvolge sempre più partecipanti, perché servono conoscenze e abilità diverse e il lavoro va suddiviso. Un **ruolo** definisce:

- Le **competenze/abilità** richieste a chi lo ricopre
- Le **attività** a cui deve partecipare chi ha quel ruolo
- Le **responsabilità** attribuite a chi ha quel ruolo

Esempi di ruoli tipici: Project Manager, Software Architect, Developer, Tester, UX/UI Designer, Analyst...

ATTENZIONE, PUNTO D'ESAME L'associazione tra **ruoli** e **persone** non è 1 a 1: una singola persona può ricoprire più ruoli contemporaneamente (specie nei team piccoli), oppure più persone possono condividere lo stesso ruolo (specie nei team grandi). In più, questa associazione **può cambiare nel corso del progetto** — non è fissata una volta per tutte all'inizio.

1.3 Attività (Task)

Un'**attività** è un'unità di lavoro assegnata a un ruolo, che riceve input e produce output significativi. Input e output possono essere di natura diversa:

- Documenti (es. specifica dei requisiti)
- Codice
- Modelli del sistema da sviluppare (es. diagrammi di progettazione)

Come già anticipato, un'attività può iniziare solo se sono soddisfatte le sue **entry conditions**, e si considera conclusa solo quando sono soddisfatte le sue **exit conditions**.

1.4 Linee guida

Le **linee guida** sono indicazioni su *come* devono essere svolte le attività, non su cosa produrre. Alcuni esempi:

- **Buone pratiche** — es. tutti gli sviluppatori devono usare uno stile di codifica uniforme
- **Priorità** — es. correggere prima i bug critici rispetto agli altri
- **Errori comuni da evitare** — es. non lasciare la documentazione disallineata rispetto al codice

1.5 A cosa serve un processo software esplicito

La domanda che si pone la lezione è diretta: **abbiamo davvero bisogno di un processo software?** La risposta è che, in realtà, **un processo lo si usa sempre**, anche senza rendersene conto. La vera differenza è tra:

- Usare in modo **inconsapevole** un processo non definito chiaramente
- Seguire in modo **consapevole** un processo definito esplicitamente, in maniera chiara e precisa

Definire esplicitamente il processo porta tre vantaggi concreti:

- Migliora la **comprensione** delle attività da eseguire da parte del team
- Rende più chiare **responsabilità, aspettative e risultati attesi**
- Rende più facile **misurare e controllare** l'avanzamento del progetto

SERVE ANCHE PER UN PROGETTO PICCOLO (1-2 PERSONE)? La slide pone questa domanda apposta come spunto di riflessione. La risposta ragionevole è: **sì, anche se in forma più leggera**. Anche un team minuscolo trae beneficio dall'avere chiari almeno gli output attesi di ogni fase e un minimo di disciplina (es. quando considerare "finita" una funzionalità), perché è proprio l'assenza di queste regole — anche in piccolo — a generare i problemi di comunicazione e di qualità visti nella Lezione 1 (Chaos Report, cowboy programmers). Un processo esplicito, però, va sempre **calibrato** sulla dimensione del progetto: un processo troppo pesante per un team di due persone sarebbe controproducente quanto l'assenza totale di processo.

1.6 Modelli di Processo Software

DEFINIZIONE Un **modello di processo software** è una descrizione **astratta** di un processo software: non riporta tutti i dettagli, ma si concentra su alcuni aspetti.

Ad esempio, un modello di processo può mettere in evidenza le caratteristiche comuni a un insieme di processi simili — come *l'ordine* in cui devono essere svolte determinate attività — trascurando dettagli specifici di ciascun processo reale (come le regole precise da seguire durante il testing).

UN'ANALOGIA UTILE Un modello di processo sta a un processo software reale un po' come una ricetta generica ("prima soffriggi, poi cuoci, poi impiatta") sta alla ricetta dettagliata di un piatto specifico: la ricetta generica indica l'ordine e le fasi principali, ma non specifica ogni singolo ingrediente o tempo di cottura — quei dettagli li aggiunge chi adatta il modello al progetto concreto.

2 · Modelli di processo tradizionali

2.1 Il modello più semplice: Code-and-fix



(con ritorno da Compile/Unit test verso Debug → Code, ogni volta che emerge un "Problem")

Il **Code-and-fix model** è il modello di processo più elementare possibile: si scrive codice, si compila, si testa; se emerge un problema si fa debug e si torna a scrivere codice, altrimenti si rilascia. I primi progetti software seguivano (implicitamente) questo modello. È adatto solo a progetti molto semplici: appena la complessità cresce, serve aggiungere altre attività — a partire da un'analisi dei requisiti e una progettazione fatte *prima* di scrivere codice, invece di procedere per tentativi.

2.2 Le attività principali di un processo

a) Analisi dei requisiti

Obiettivo: determinare **quali funzioni** deve svolgere il software e **quali vincoli** vanno rispettati (è lo stesso concetto di requisiti funzionali/non funzionali e design constraint visto nella Lezione 1). I requisiti vanno di norma **documentati** e **validati dal committente**, cioè da chi ha commissionato il software: senza questa validazione formale, si rischia di scoprire solo a lavori avanzati che i requisiti erano stati fraintesi.

b) Progettazione (Design)

Obiettivo: definire la **struttura** del software, tipicamente costruendo diversi modelli, a diversi livelli di dettaglio:

- **Architettura** — struttura di alto livello del sistema
- **Interfacce** — punti di comunicazione tra i componenti
- **Componenti** — unità individuali all'interno del sistema
- **Strutture dati**
- **Algoritmi**

c) Implementazione

Obiettivo: trasformare il design in un programma funzionante. Due punti da ricordare:

- Spesso **non c'è una netta separazione** tra implementazione e i livelli più bassi di progettazione (molte decisioni di dettaglio si prendono proprio mentre si scrive il codice)
- **Non è un'attività puramente meccanica**: richiede di prendere continuamente decisioni critiche, non solo "tradurre" un progetto già completo

d) Integrazione e testing

Integrazione: combinare i diversi componenti per formare un sistema completo. **Testing:** verificare sperimentalmente che il software funzioni come dovrebbe. Esistono diversi **livelli di testing**, ciascuno con uno scopo e responsabili diversi:

Livello	Cosa verifica	Chi lo esegue tipicamente
Unit testing	I singoli componenti, isolatamente	Chi sviluppa il componente
Integration testing	Che componenti diversi funzionino bene insieme	Il team di sviluppo
System testing	Il sistema completo	Un gruppo di tester dedicato
Acceptance testing	Che il sistema soddisfi le esigenze reali	Il committente

PERCHÉ LA PROGRESSIONE HA SENSO Questi quattro livelli sono ordinati "dal piccolo al grande" apposta: si verifica prima che ogni singolo mattoncino funzioni da solo (unit), poi che i mattoncini si incastrino correttamente tra loro (integration), poi che l'edificio intero regga (system), e infine che sia davvero quello che il committente voleva abitare (acceptance). Saltare un livello rende molto più difficile capire *dove* si trova un problema quando emerge più avanti.

e) Rilascio e manutenzione

- **Deployment** — installazione del software nell'ambiente di produzione (esercizio reale)

Attività post-deployment:

- **Data migration** — trasferimento dati da sistemi preesistenti
- **Bug fixing** — identificazione e rimozione di difetti

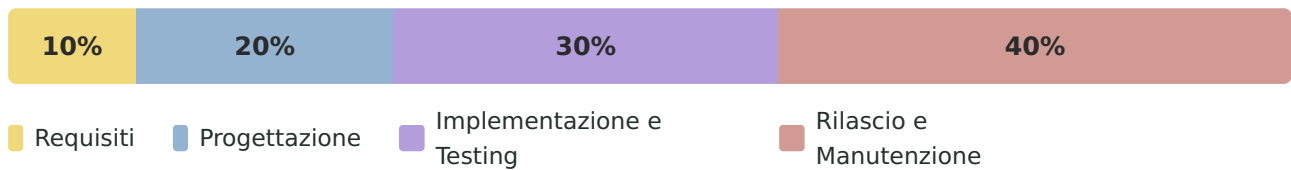
Tipi di manutenzione:

- **Adattativa** — modifiche per adattare il sistema a cambiamenti nell'ambiente in cui opera (es. un nuovo sistema operativo, una nuova normativa)
- **Evolutiva** — aggiunta di nuove funzioni o miglioramento di quelle esistenti

APPROFONDIMENTO — LE ALTRE CATEGORIE CLASSICHE DI MANUTENZIONE Nella letteratura di riferimento (Sommerville e la classificazione di Swanson) si distinguono in genere **quattro** tipi di manutenzione, non solo due: oltre ad *adattativa* ed *evolutiva* viste a lezione, si parla anche di manutenzione **correttiva** (corrisponde di fatto al "bug fixing" già visto come attività post-deployment) e **perfettiva** (migliorare qualità interna del codice — leggibilità, manutenibilità — senza cambiarne il comportamento esterno). Utile per inquadrare meglio il bug fixing all'interno dello schema generale delle manutenzioni.

2.3 Il costo delle attività

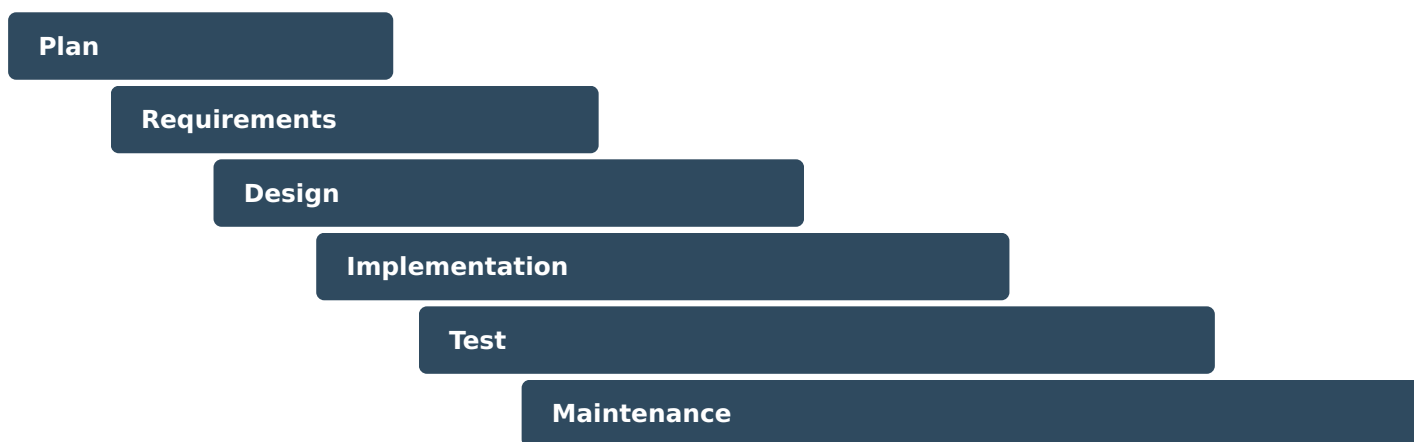
Una domanda naturale: quanto incidono le diverse attività sul **costo totale** di un sistema software nel corso della sua intera vita (non solo nello sviluppo iniziale)? I dati mostrati a lezione danno questa ripartizione indicativa:



IL DATO PIÙ IMPORTANTE DA RICORDARE Il rilascio e la manutenzione, da soli, pesano quanto tutte le altre fasi messe insieme (40% contro 60% ripartito su tre fasi, ma su un arco di tempo molto più lungo). Questo è coerente con quanto visto nella Lezione 1 a proposito del costo crescente degli errori scoperti tardi: un sistema non "finisce" quando viene rilasciato, anzi è lì che comincia la fase più costosa del suo ciclo di vita.

2.4 Modello a cascata (Waterfall)

È il **primo modello di processo** a essere stato formalmente definito. Le attività si svolgono **sequenzialmente**: ciascuna riceve come input l'output della precedente.



Caratteristiche

- Ogni attività deve **terminare prima** che inizi la successiva
- Gli output di ogni attività sono **documentati, verificati e approvati**
- Ogni attività si basa **esclusivamente** sugli output della precedente

Vantaggi

- Struttura chiara, con fasi, input e output ben definiti
- Semplifica la divisione del lavoro, il monitoraggio e la gestione del progetto
- Consente di descrivere lo stato di avanzamento in modo preciso, invece del vago "quasi finito"

Svantaggi

- Il feedback dagli utenti arriva solo alla fine
- I cambiamenti sono costosi, specialmente sui requisiti
- Documentazione "pesante"
- Se il progetto si ferma prima della fine, si rischia di non avere risultati intermedi utilizzabili

Applicabilità

- Progetti di **grandi dimensioni**, sviluppati da team altrettanto grandi
- Progetti in cui è importante poter **verificare formalmente** gli output di ciascuna attività
- Progetti in cui i **requisiti non cambiano** nel tempo

Varianti: Waterfall con feedback

Per limitare il difetto principale del modello puro (nessun ritorno indietro), sono state proposte varianti che introducono **canali di feedback**:

- **Waterfall con feedback** — un canale di ritorno verso la primissima fase (requirements), utile per correzioni maggiori
- **Waterfall con feedback ad ogni fase** — ogni fase può restituire un feedback diretto alla fase immediatamente precedente, rendendo il modello leggermente più flessibile

Restano comunque modelli fondamentalmente **sequenziali**: il feedback è un correttivo, non cambia la logica di fondo "una fase alla volta".

2.5 Modelli di processo iterativi

A differenza del Waterfall, i modelli iterativi sviluppano il sistema attraverso **più iterazioni**, ciascuna delle quali ripete al proprio interno le diverse attività (analisi dei requisiti, progettazione, implementazione, integrazione e testing).

Vantaggi

- Ideali quando i requisiti **cambiano nel tempo** o non sono interamente definiti all'inizio
- Ogni iterazione dà un **feedback** dagli stakeholder (utenti e altri soggetti interessati)
- I problemi vengono individuati e risolti **prima**

Esempi di modelli iterativi

- Modello incrementale
- Modello a spirale
- Unified Process

2.6 Modello incrementale

Il progetto è diviso in **incrementi**, rilasciati in momenti successivi (**release**). Prima si definiscono una panoramica dei requisiti (*requirements outline*), un'architettura di massima (*architecture design*) e si assegnano i requisiti ai vari incrementi; poi ogni incremento viene sviluppato come un piccolo progetto a sé.



...lo stesso ciclo si ripete per Release 2, Release 3, ..., Release n

- Ogni incremento contiene un **sottoinsieme crescente** delle funzionalità dell'intero sistema
- Ogni incremento è un vero e proprio **sotto-progetto**, in cui si svolgono tutte le attività normalmente previste in un progetto

Vantaggi

- Il cliente ottiene **velocemente** un sottoinsieme funzionante del sistema
- Le prime release aiutano a capire meglio i requisiti per quelle successive
- Riduzione del rischio di **fallimento completo** del progetto
- Le funzionalità più importanti sono realizzate per prime, e quindi testate più a lungo

Svantaggi e applicabilità

- **Svantaggio:** più difficile sfruttare il parallelismo in team grandi
- **Applicabilità:** progetti di dimensioni medio/piccole, di lunga durata, con requisiti che cambiano nel tempo

WATERFALL VS INCREMENTALE, IL CONFRONTO CHE CONTA PER L'ESAME Il Waterfall ottimizza per **controllo e prevedibilità** (requisiti stabili, verifica formale, grandi team): il prezzo è avere feedback reale solo alla fine. Il modello incrementale ottimizza per **adattabilità e riduzione del rischio** (requisiti che cambiano, feedback continuo): il prezzo è una minore efficienza nel parallelizzare il lavoro su team molto grandi. Non esiste un modello "migliore in assoluto" — coerentemente con la conclusione della Lezione 1: ogni progetto è unico.

3 · IA Generativa e Processo di Sviluppo Software

3.1 "No Silver Bullet"

Nel corso degli anni, molte innovazioni sono state presentate come **soluzioni decisive** alle difficoltà dello sviluppo software: programmazione strutturata e orientata agli oggetti, IDE visuali, sviluppo basato su componenti, linguaggi/piattaforme come Java e .NET, XML e servizi Web (SOA), architetture a microservizi, e — più di recente — IA generativa e agenti di sviluppo software.

LA TESI DI BROOKS (1986) Ciascuna di queste innovazioni può offrire benefici reali, ma **nessuna elimina** la complessità intrinseca dello sviluppo software. Brooks distingue due componenti della complessità:

- **Complessità accidentale** — legata agli strumenti e alle tecnologie usate per realizzare il software
- **Complessità essenziale** — intrinseca al problema da risolvere, ai requisiti e alle interazioni tra le parti del sistema

Le nuove tecnologie possono ridurre la **complessità accidentale** (rendono più veloce o più comodo scrivere codice), ma **non eliminano** quella **essenziale** (capire bene il problema, gestire le interazioni tra requisiti, prendere decisioni di compromesso restano difficili comunque). Non esiste quindi una "pallottola d'argento": restano sempre necessari metodo, competenze e verifica.

ESEMPIO PER FISSARE IL CONCETTO Un IDE moderno con autocompletamento riduce la complessità *accidentale* di scrivere una funzione (meno tempo perso a ricordare la sintassi esatta). Ma se il vero problema è che due requisiti del cliente sono in conflitto tra loro, nessun IDE — per quanto sofisticato — può risolvere da solo quella complessità *essenziale*: serve capire il dominio, parlare con gli stakeholder, prendere una decisione di compromesso.

3.2 IA-Assisted Software Engineering

L'IA generativa può fornire supporto lungo tutto il processo di sviluppo:

Fase	Come può aiutare l'IA generativa
Requisiti	Proporre formulazioni, individuare ambiguità
Progettazione	Suggerire alternative e discuterne i compromessi (trade-off)
Implementazione	Generare, spiegare e modificare codice

Testing

Proporre casi di test, supportare l'analisi degli errori

Manutenzione

Aiutare a comprendere e aggiornare software esistente

IL LIMITE FONDAMENTALE Definire **gli obiettivi**, valutare le soluzioni proposte e garantirne l'adeguatezza **richiede comunque competenza e responsabilità umana**. L'IA generativa opera bene dentro la complessità accidentale, ma non sostituisce il giudizio necessario per affrontare la complessità essenziale del problema.

3.3 Debito tecnico e debito cognitivo

L'uso dell'IA può accelerare la produzione del software, ma può anche favorire l'accumulo di due forme di "debito":

DEBITO TECNICO Scelte di implementazione e progettazione che rendono più costose le modifiche future: es. codice duplicato, dipendenze superflue, soluzioni incoerenti con l'architettura esistente.

DEBITO COGNITIVO Il divario tra come funziona realmente il sistema e la comprensione che ne ha il team: si accumula quando si integra codice generato senza comprenderlo davvero.

DA RICORDARE PER L'ESAME "Il tempo risparmiato all'inizio può essere pagato in seguito, con gli interessi, per comprendere, correggere e modificare il software." È la stessa logica del **costo crescente degli errori** vista nella Lezione 1: accelerare la produzione senza comprensione sposta il costo più avanti nel tempo, dove è più alto.

3.4 Il ruolo dell'ingegnere del software

Nel contesto dell'IA generativa, il ruolo dell'ingegnere del software diventa quello di **guidare e valutare criticamente** l'uso dell'IA, non di delegargli le decisioni:

- **Chiarire** il problema, i requisiti e le ambiguità
- **Valutare** le alternative e motivare le scelte
- **Comprendere** il codice prima di integrarlo
- **Verificare** correttezza, sicurezza e coerenza con il progetto
- **Assumersi la responsabilità** delle soluzioni adottate

FRASE CHIAVE DELLA LEZIONE "Saper chiedere una soluzione non basta: occorre saperla comprendere, valutare e verificare."

Riferimento del corso: I. Sommerville, *Ingegneria del Software*, 10ª ed., Pearson — Capitolo 2 (tranne 2.3 e 2.4).

4 · Riepilogo per l'esame

Mappa concettuale in breve

Processo software → insieme di regole per svolgere, coordinare e controllare le attività di un progetto. Definisce attività (con input/output ed entry/exit condition), **ruoli** (competenze, attività, responsabilità — associazione non 1 a 1 con le persone) e **linee guida**. Un processo esplicito migliora comprensione, chiarezza di responsabilità e controllo dell'avanzamento — utile anche nei piccoli progetti, in forma proporzionata.

Modello di processo → descrizione astratta di un processo, che ne evidenzia alcune caratteristiche (es. l'ordine delle attività) senza tutti i dettagli.

Code-and-fix → il modello più semplice (problem → code → compile → test → release, con ritorno a debug); adatto solo a progetti molto semplici.

Attività principali → requisiti (validati dal committente) → progettazione (architettura, interfacce, componenti, dati, algoritmi) → implementazione (non meccanica) → integrazione e testing (unit → integration → system → acceptance) → rilascio e manutenzione (adattativa, evolutiva, + correttiva/perfettiva). Il 40% del costo totale è nel rilascio/manutenzione.

Waterfall → sequenziale, fasi ben verificate, ma feedback solo alla fine e cambiamenti costosi; adatto a progetti grandi con requisiti stabili. Le varianti aggiungono canali di **feedback** tra le fasi, restando comunque sequenziali.

Modelli iterativi (es. incrementale) → più iterazioni, ciascuna con tutte le attività; adatti a requisiti che cambiano; riducono il rischio di fallimento totale, ma sono meno efficienti da parallelizzare su team grandi.

No Silver Bullet (Brooks 1986) → nessuna tecnologia, IA generativa inclusa, elimina la **complessità essenziale** del software (solo quella **accidentale**). L'IA aiuta in ogni fase del processo, ma può generare **debito tecnico** e **debito cognitivo**. Il ruolo dell'ingegnere resta chiarire, valutare, comprendere, verificare e assumersi la responsabilità delle scelte.

Domande di autoverifica

1. Quali sono i cinque/sei elementi che un processo software deve definire?

→ Le attività da svolgere; input e output di ciascuna attività; le entry conditions e le exit conditions di ogni attività; i ruoli da ricoprire; le linee guida per lo svolgimento del progetto.

2. È vero che l'associazione tra ruoli e persone in un progetto è sempre 1 a 1?

→ No: una persona può ricoprire più ruoli e più persone possono condividere lo stesso ruolo; inoltre questa associazione può cambiare nel corso del progetto.

3. Che differenza c'è tra usare un processo software e usarne uno esplicito?

→ Un processo, anche informale, lo si usa sempre. La differenza è tra farlo inconsapevolmente (processo non definito chiaramente) o consapevolmente (processo definito in modo esplicito e preciso), il che migliora comprensione, chiarezza delle responsabilità e possibilità di misurare l'avanzamento.

4. Cos'è un modello di processo software, rispetto a un processo software reale?

→ È una descrizione astratta che mette in evidenza solo alcuni aspetti comuni (es. l'ordine delle attività), trascurando i dettagli specifici di un singolo processo reale.

5. Perché il Code-and-fix model non è adatto a progetti non banali?

→ Perché non prevede fasi esplicite di analisi dei requisiti e progettazione prima di scrivere codice: funziona solo quando il problema è così semplice da poter procedere per tentativi diretti su compile/test.

6. Elenca i quattro livelli di testing e chi li esegue tipicamente.

→ Unit testing (chi sviluppa il componente), integration testing (il team), system testing (un gruppo di tester dedicato), acceptance testing (il committente).

7. Come si ripartisce, indicativamente, il costo totale di un sistema software tra le diverse attività?

→ Requisiti 10%, progettazione 20%, implementazione e testing 30%, rilascio e manutenzione 40%: quest'ultima fase da sola pesa quanto tutte le altre messe insieme.

8. Quali sono le caratteristiche principali del modello a cascata (Waterfall)?

→ Le attività si svolgono sequenzialmente: ogni attività termina prima che inizi la successiva, gli output sono documentati/verificati/approvati, e ogni attività si basa esclusivamente sull'output della precedente.

9. In quali contesti è più indicato il Waterfall, e in quali il modello incrementale?

→ Waterfall: progetti grandi, con team grandi, requisiti stabili, dove serve verifica formale di ogni fase. Incrementale: progetti medio/piccoli, di lunga durata, con requisiti che cambiano nel tempo, dove serve ridurre il rischio e avere feedback frequente.

10. Cosa cambia nelle varianti "Waterfall con feedback" rispetto al Waterfall puro?

→ Si aggiungono canali di ritorno tra le fasi (verso i requisiti, oppure verso la fase immediatamente precedente ad ogni step), ma il modello resta fondamentalmente sequenziale: il feedback è un correttivo, non introduce iterazioni complete come nei modelli iterativi.

11. Cosa intende Brooks con "complessità accidentale" e "complessità essenziale", e cosa comporta per l'IA generativa?

→ La complessità accidentale dipende dagli strumenti/tecnologie usati e può essere ridotta da nuove tecnologie (inclusa l'IA generativa); la complessità essenziale è intrinseca al problema, ai requisiti e alle loro interazioni, e nessuna tecnologia — IA compresa — può eliminarla: restano necessari comprensione del dominio e giudizio umano.

12. Cosa sono il debito tecnico e il debito cognitivo, e perché sono rilevanti quando si usa l'IA generativa?

→ Il debito tecnico è l'insieme di scelte implementative/progettuali (es. duplicazioni, dipendenze superflue) che rendono più costose le modifiche future; il debito cognitivo è il divario tra come funziona il sistema e quanto il team lo comprende davvero. L'IA può accelerare la produzione ma anche far accumulare entrambi i debiti se il codice generato viene integrato senza essere compreso.

13. Qual è, secondo la lezione, il ruolo dell'ingegnere del software nell'uso dell'IA generativa?

→ Guidare e valutare criticamente l'uso dell'IA: chiarire problema e requisiti, valutare le alternative, comprendere il codice prima di integrarlo, verificarne correttezza/sicurezza/coerenza, e assumersi la responsabilità delle soluzioni adottate — sapere chiedere una soluzione all'IA non basta, occorre saperla comprendere, valutare e verificare.